Generalized linear models

Logistic regression and count models notes

Foundations of Regression and Predictive Modeling

Table of contents

Why logistic regression exists . . . 1
Intro case: NBA point spreads . . . 2
Fitting the logistic model . . . 3
Odds ratios . . . 4
Complex case: airline connections . . . 4
Start with arrival delay . . . 5
Add scheduled connection time . . . 6
Hub and terminal changes . . . 7
Add the interaction . . . 8
Reading the interaction . . . 9
What to carry forward . . . 10
Part two: count outcomes . . . 10
The dock rentals case . . . 10
The turbine case: offsets . . . 12
The turbine case: overdispersion and the negative binomial . . . 13
The GLM family . . . 15

Why logistic regression exists

Ordinary regression expects a numerical response. Logistic regression is for a binary response: win/loss, made/missed, yes/no, churned/stayed, converted/did not convert.

The key problem is that probabilities must stay between 0 and 1. A straight-line model for probability can wander outside that range. Logistic regression solves this by modeling the log odds:

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_i.$$

The left side can be any real number. The corresponding probability stays between 0 and 1.

Intro case: NBA point spreads

Sportsbooks publish a point spread for NBA games. Here `HomeSpread` is the spread in favor of the home team. Negative means the visitor is favored. Positive means the home team is favored.

```
nba_games <- read_csv(file.path(module_dir, "nba_home_spreads.csv")) |>
  mutate(HomeWin = factor(HomeWin, levels = c("Visitor win", "Home win")))
```

The question is:

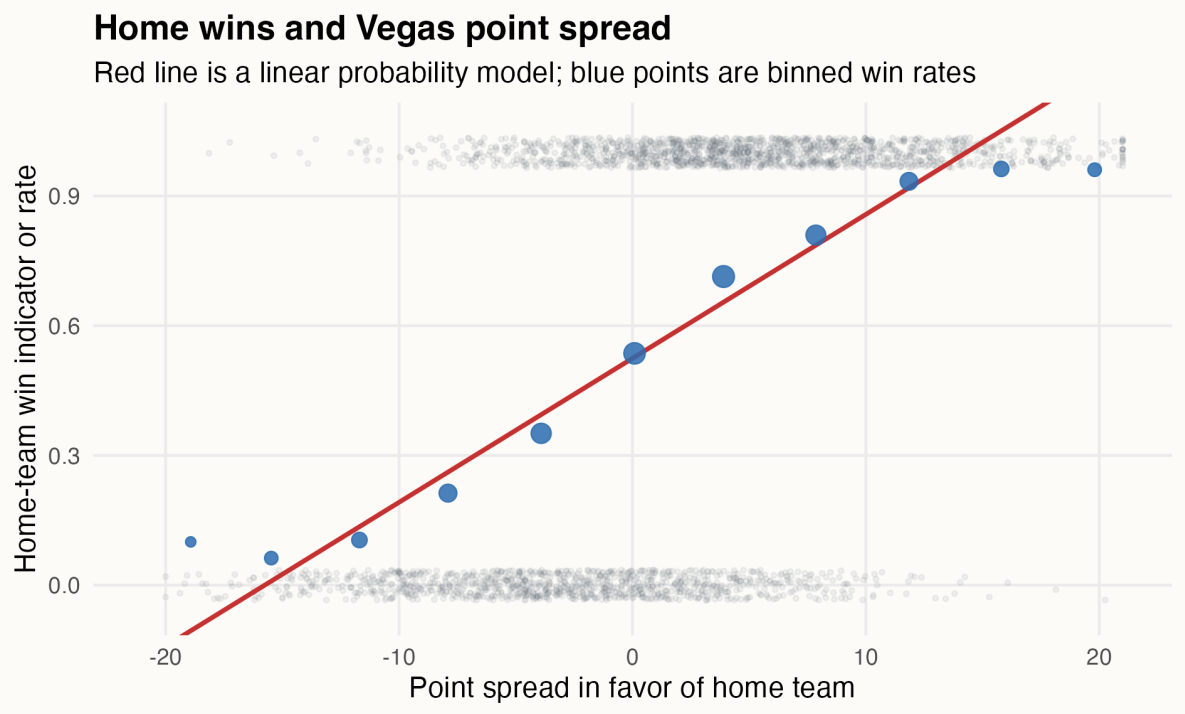
> How does the point spread translate into home-team win probability?

The linear probability model is tempting:

```
nba_lpm <- lm(as.integer(HomeWin == "Home win") ~ HomeSpread, data = nba_games)
tidy(nba_lpm)
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)   0.525    0.00846      62.0 0
2 HomeSpread    0.0333   0.00106      31.3 4.46e-183
```

But it uses a straight line for a probability.



Fitting the logistic model

Fit logistic regression with `glm()` and `family = binomial()`:

```
nba_fit <- glm(
  HomeWin ~ HomeSpread,
  data = nba_games,
  family = binomial()
)

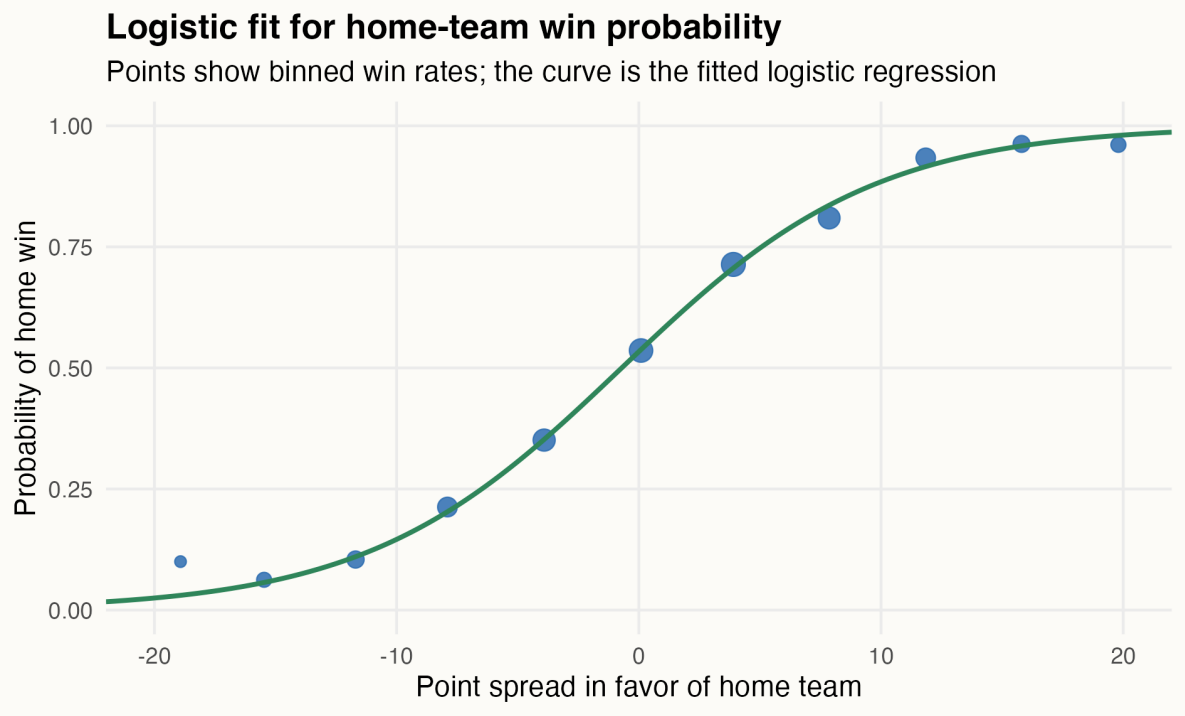
tidy(nba_fit)
```

```
# A tibble: 2 x 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)    0.135   0.0475       2.84 4.48e-  3
2 HomeSpread     0.190   0.00825     23.0  1.95e-117
```

A note on what `glm()` is doing under the hood. `lm()` chooses coefficients to minimize the sum of squared residuals, and that recipe does not apply here: the outcomes are 0s and 1s while the model predicts probabilities. `glm()` instead chooses coefficients by **maximum likelihood** — find the coefficients that give the highest probability to the outcomes actually observed. Least squares is the special case of this principle for a numerical response with normal errors, and the standard errors and intervals in the output come from the same likelihood machinery. Nothing about how you *read* the output changes.

The fitted model is approximately

$$\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = 0.135 + 0.190 \cdot \text{HomeSpread}.$$



Odds ratios

The coefficient is on the log-odds scale. Exponentiating gives an odds ratio:

```
exp(coef(nba_fit)["HomeSpread"])
```

```
HomeSpread
  1.209383
```

Here the odds ratio is about 1.21. So each additional point of home-team spread multiplies the odds of a home win by about 1.21.

This does not mean each point adds 21 percentage points to the win probability. Logistic regression is additive on the log-odds scale, not on the probability scale.

Complex case: airline connections

An airline studies passenger connections through six hubs: ATL, CLT, DFW, DEN, ORD, and IAH.

Each row is one passenger connection.

```
connections <- read_csv(file.path(module_dir, "airline_connections.csv")) |>
  mutate(
    MadeConnection = factor(MadeConnection, levels = c("Missed connection", "Made connection")),
    Hub = factor(Hub, levels = c("ATL", "CLT", "DFW", "DEN", "ORD", "IAH")),
    TerminalChange = factor(TerminalChange, levels = c("Same terminal", "Terminal change"))
  )
```

The variables are:

- `MadeConnection`: made or missed the connection,
- `ScheduledConnectionMinutes`: scheduled time between flights,
- `ArrivalDelayMinutes`: delay of the inbound flight,
- `Hub`: connecting airport,
- `TerminalChange`: same terminal or terminal change.

The question is:

> Which connections are likely to be made?

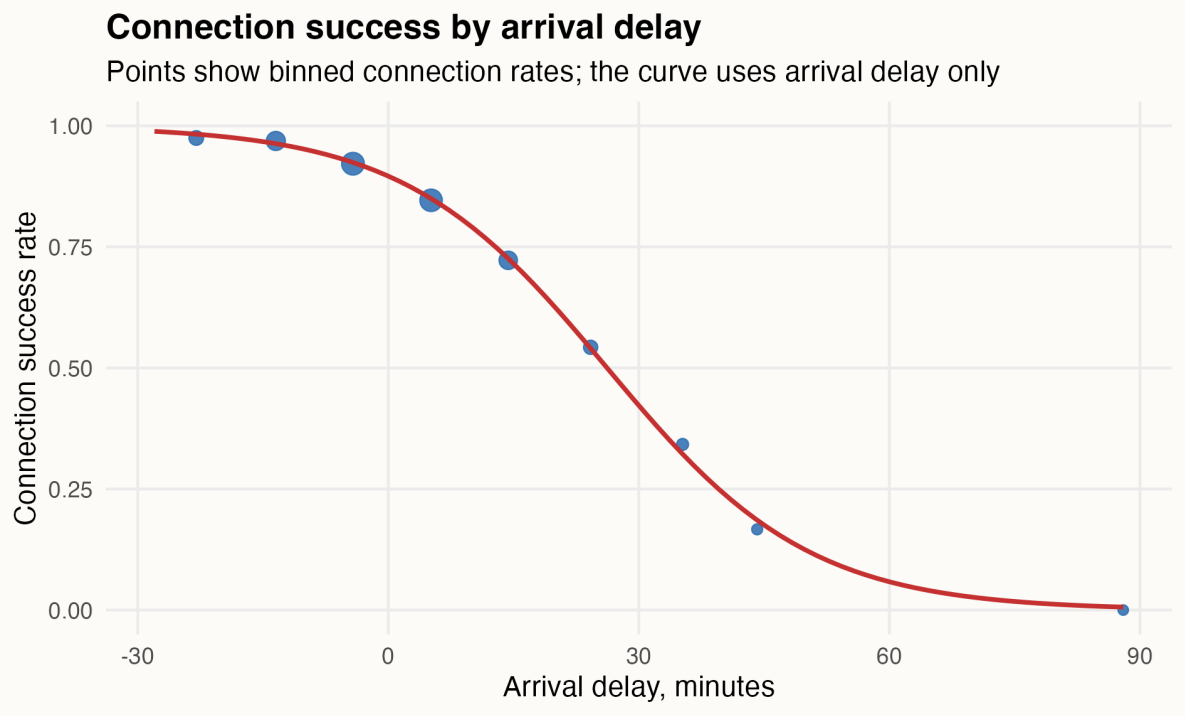
Start with arrival delay

The simplest model uses arrival delay only:

```
delay_only <- glm(
  MadeConnection ~ ArrivalDelayMinutes,
  data = connections,
  family = binomial()
)

tidy(delay_only)
```

```
# A tibble: 2 x 5
  term                estimate std.error statistic   p.value
  <chr>                  <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)            2.15     0.0429       50.3 0
2 ArrivalDelayMinutes   -0.0823   0.00352     -23.4 6.21e-121
```



Arrival delay matters, but delay alone is incomplete. A 20-minute delay is different when the original itinerary had a 35-minute connection versus a 95-minute connection.

Add scheduled connection time

Now include both the scheduled buffer and the delay:

```
connection_main <- glm(
  MadeConnection ~ ScheduledConnectionMinutes + ArrivalDelayMinutes +
    Hub + TerminalChange,
  data = connections,
  family = binomial()
)

tidy(connection_main)
```

```
# A tibble: 9 x 5
  term                       estimate std.error statistic   p.value
  <chr>                         <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)                  -2.00     0.180      -11.1  7.26e- 29
2 ScheduledConnectionMinutes    0.0861   0.00339     25.4  1.14e-142
3 ArrivalDelayMinutes          -0.101    0.00425    -23.8  1.11e-125
4 HubCLT                       -0.285    0.143       -1.99 4.62e-  2
5 HubDFW                        0.193    0.141        1.37 1.71e-  1
6 HubDEN                       -0.188    0.146       -1.29 1.98e-  1
7 HubORD                       -0.960    0.130       -7.37 1.74e- 13
```

```
8 HubIAH                          -0.588    0.144       -4.09 4.23e-  5
9 TerminalChangeTerminal change   -0.489    0.0900      -5.44 5.35e-  8
```

Ten-minute odds ratios are easier to read:

```
coef(connection_main)[c("ScheduledConnectionMinutes", "ArrivalDelayMinutes")] |>
  (\(x) exp(10 * x))()
```

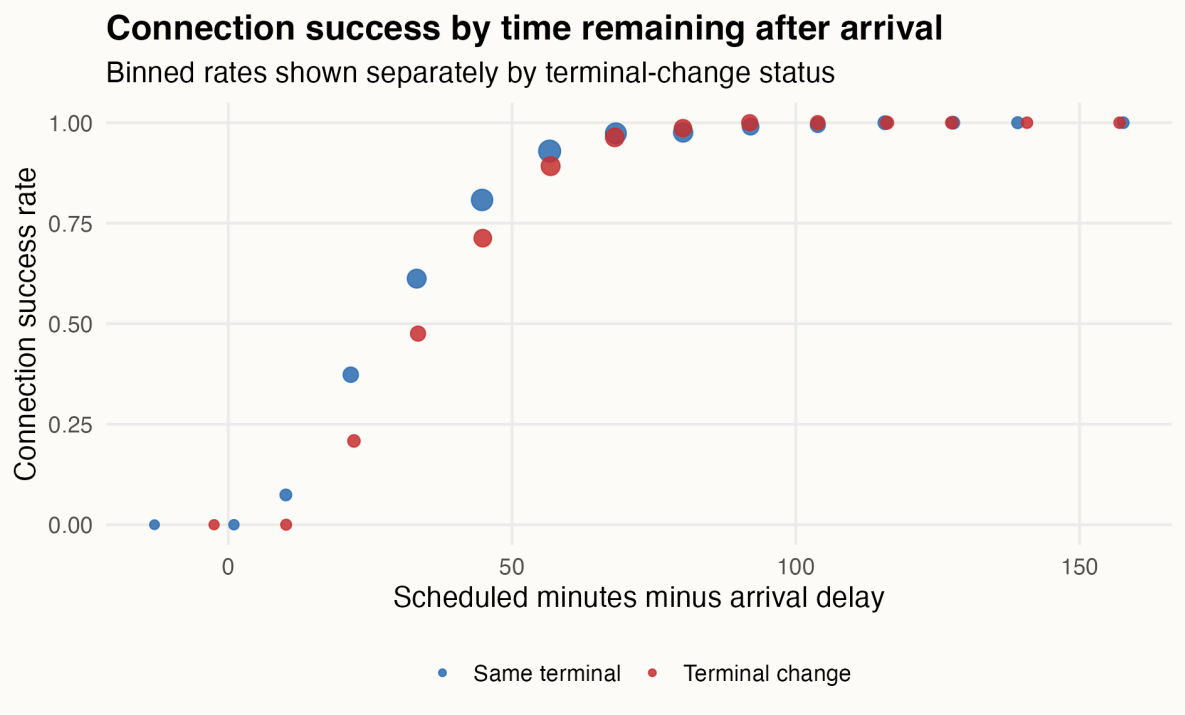
```
ScheduledConnectionMinutes        ArrivalDelayMinutes
                 2.3654721                  0.3632114
```

In this model, 10 more scheduled connection minutes multiply the odds of making the connection by about 2.4. Ten more minutes of arrival delay multiply the odds by about 0.36.

Hub and terminal changes

A terminal change is not the same at every hub. DFW has the Skylink train, and most gate pairs are reachable quickly. ATL has the Plane Train and a regular concourse layout. ORD and IAH can impose a larger terminal-change penalty.

The first visual clue is that terminal-change connections trail same-terminal connections, but the gap is not the same everywhere.



Add the interaction

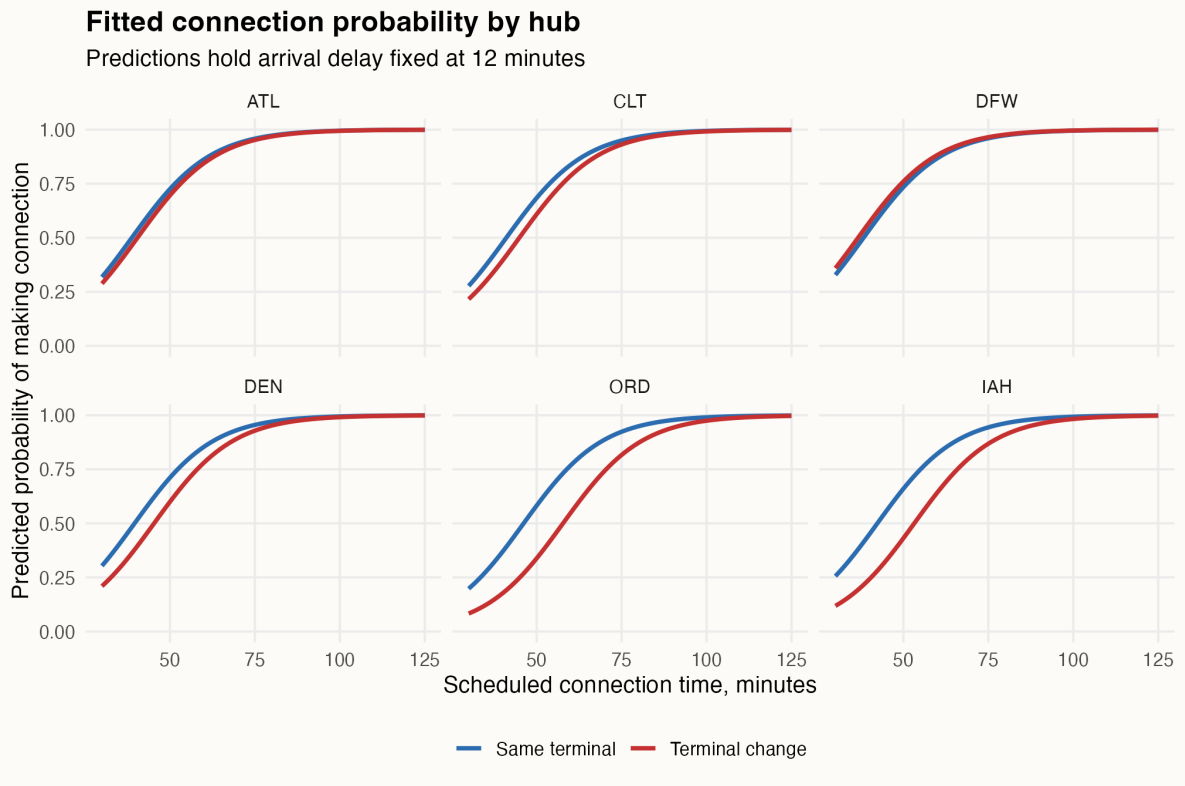
Use a hub-by-terminal-change interaction:

```
connection_interaction <- glm(
  MadeConnection ~ ScheduledConnectionMinutes + ArrivalDelayMinutes +
    Hub * TerminalChange,
  data = connections,
  family = binomial()
)

tidy(connection_interaction)
```

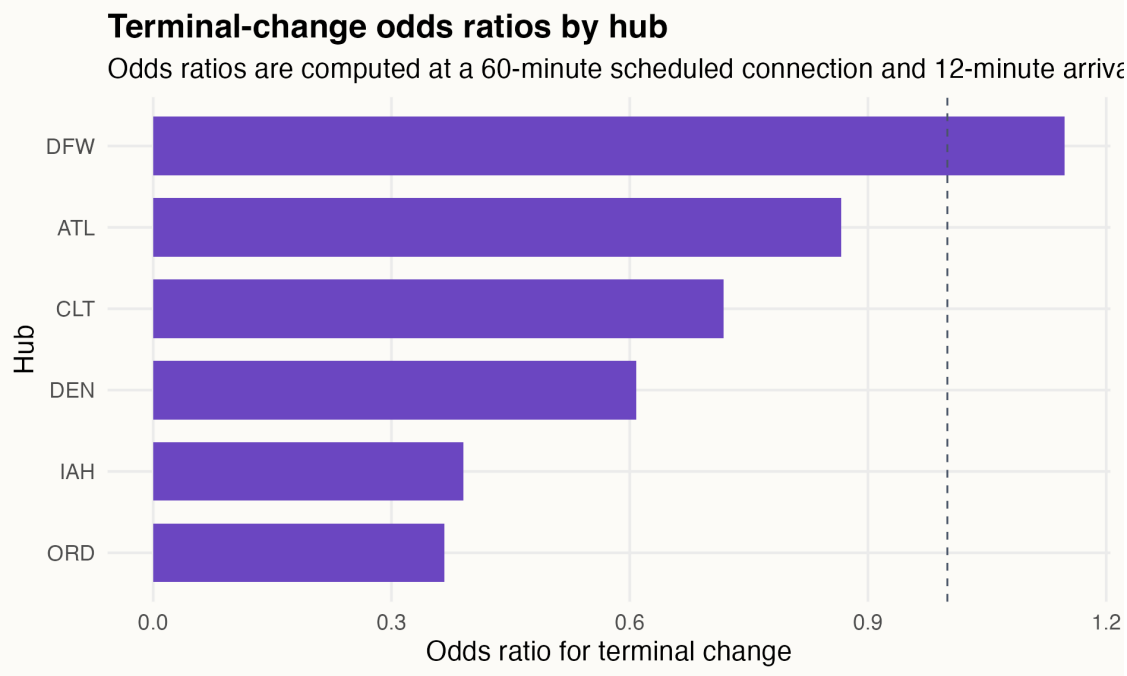
```
# A tibble: 14 x 5
   term                                    estimate std.error statistic   p.value
   <chr>                                      <dbl>     <dbl>     <dbl>     <dbl>
 1 (Intercept)                              -2.14     0.191     -11.2   3.20e- 29
 2 ScheduledConnectionMinutes                0.0866   0.00341    25.4   4.65e-142
 3 ArrivalDelayMinutes                      -0.102    0.00426   -23.9   5.77e-126
 4 HubCLT                                   -0.197    0.164      -1.20  2.28e-  1
 5 HubDFW                                    0.0455   0.176       0.258 7.96e-  1
 6 HubDEN                                   -0.0687   0.177      -0.387 6.99e-  1
 7 HubORD                                   -0.638    0.164      -3.89  9.97e-  5
 8 HubIAH                                   -0.306    0.179      -1.70  8.83e-  2
 9 TerminalChangeTerminal change            -0.143    0.204      -0.702 4.83e-  1
10 HubCLT:TerminalChangeTerminal change     -0.188    0.347      -0.542 5.88e-  1
11 HubDFW:TerminalChangeTerminal change      0.281    0.297       0.947 3.44e-  1
12 HubDEN:TerminalChangeTerminal change     -0.354    0.312      -1.14  2.56e-  1
13 HubORD:TerminalChangeTerminal change     -0.860    0.272      -3.16  1.56e-  3
14 HubIAH:TerminalChangeTerminal change     -0.797    0.301      -2.65  7.99e-  3
```

The model still has the same logistic form. The interaction lets the terminal-change odds ratio vary by hub.



Reading the interaction

The terminal-change penalty can be summarized as an odds ratio at a fixed scheduled connection time and arrival delay.



At a 60-minute scheduled connection and 12-minute arrival delay:

- DFW terminal change odds ratio is about 1.1 — no detectable penalty,
- ORD terminal change odds ratio is about 0.37,
- IAH terminal change odds ratio is about 0.39.

The interaction answers the real question better than one shared terminal-change coefficient.

What to carry forward

Logistic regression is a generalized linear model for binary outcomes. It keeps the linear predictor, but changes the scale:

$$\log\left(\frac{p}{1-p}\right) = \text{linear predictor}.$$

Exponentiated coefficients are odds ratios. Interactions work the same way they did before: they let an effect depend on context.

Part two: count outcomes

The other response type from the opening list is a count: how many rentals started at this dock, how many maintenance events this machine logged, how many tickets, claims, no-shows, or defects. A count is not a yes/no, and it is not an ordinary numerical measurement either. It is a non-negative integer, often small, and often skewed.

The dock rentals case

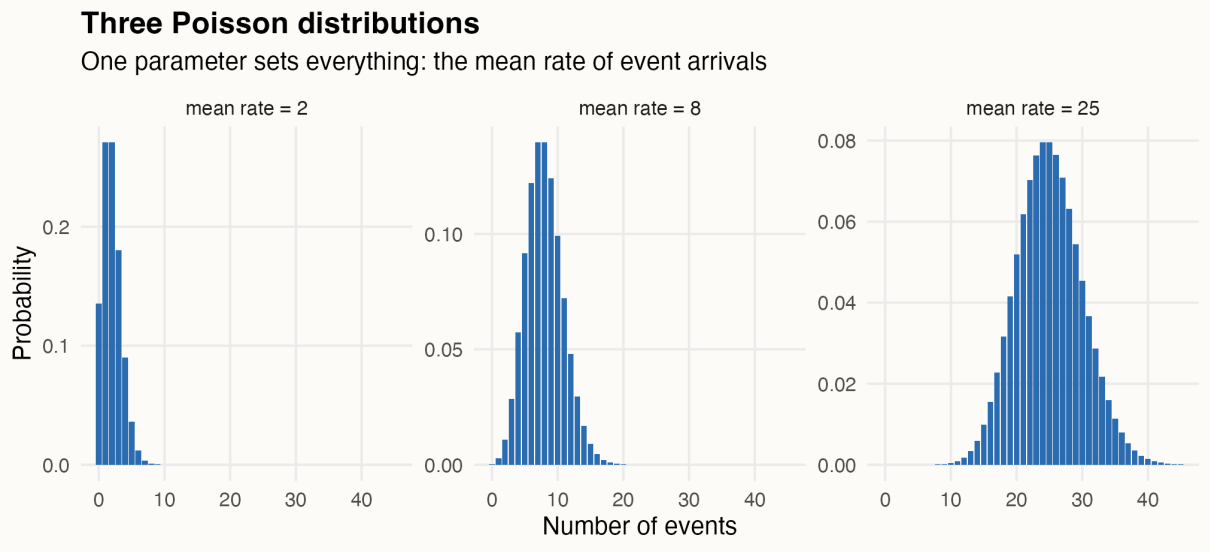
Austin's bike-share ops team restocks a busy downtown dock every morning. Stocking too few bikes loses rides; trucking too many across town wastes labor. The data are one row per day for a full year: the daily high temperature, a weekend indicator, and the number of rentals started at the dock.

```
bike_days <- read_csv(file.path(module_dir, "dock_rentals.csv"))
bike_days |> head(4)
```

```
# A tibble: 4 x 4
  Date       TempHighF Weekend Rentals
  <date>         <dbl>   <dbl>   <dbl>
1 2024-01-01        38       0       0
2 2024-01-02        41       0       1
3 2024-01-03        46       0       3
4 2024-01-04        57       0       5
```

The straight-line model `lm(Rentals ~ TempHighF + Weekend)` makes two promises the data cannot keep. It promises the same additive change everywhere — about 0.36 extra rentals per degree, whether the day averages 3 rides or 25 — and it promises constant scatter, while the observed scatter grows visibly with the level.

First, the distribution doing the work. The **Poisson distribution** is the workhorse distribution for counts, and it has exactly one parameter: the mean rate of event arrivals, μ. If events arrive independently at an average rate of μ per day, the number that actually arrive on a given day follows a Poisson distribution with mean μ. Set the rate and the whole distribution follows, including its spread — a fact that becomes a checkable promise below.



The module-02 instinct is to log the response. But the dock had days with zero rentals, and $\log(0)$ does not exist, while $\log(y+1)$ quietly changes the question. The GLM move is different: leave the counts alone and put the log on the *mean*,

$$\log(\mu_i) = \beta_0 + \beta_1 \,\text{TempHighF}_i + \beta_2 \,\text{Weekend}_i,$$

with the counts treated as Poisson around that mean. A mean of 3.2 rentals is a perfectly good number to take the log of, even on a day that happened to produce zero.

```
bike_pois <- glm(Rentals ~ TempHighF + Weekend, data = bike_days, family = poisson())
tidy(bike_pois, exponentiate = TRUE, conf.int = TRUE)
```

```
# A tibble: 3 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)    0.557   0.0819      -7.15 8.93e-13    0.474     0.653
2 TempHighF      1.04    0.00101     37.6  0           1.04      1.04
3 Weekend        1.46    0.0343      11.1  9.03e-29    1.37      1.57
```

Exponentiated coefficients are **rate ratios**, the count-world sibling of odds ratios. Each degree multiplies expected rentals by about 1.039 — a 3.9% lift — and weekends run about 46% above weekdays at any temperature. The effects are multiplicative: a warm snap adds more rides at a busy dock than at a dead one, automatically.

Because the model specifies a full count distribution, it answers inventory questions, not just trend questions. On a 90-degree Saturday the fitted mean is about 25.3 rentals, and `qpois(0.96, 25.3)` says that staging 34 bikes covers demand on about 96% of such days.

Poisson regression makes one strong promise that deserves a check: $\mathrm{Var}(Y_i) = \mu_i$. Group the days into bins by fitted mean and compare the within-bin variance to the within-bin mean, or compute the Pearson dispersion statistic — squared standardized residuals, averaged:

```
sum(residuals(bike_pois, type = "pearson")^2) / df.residual(bike_pois)
```

```
[1] 0.9549859
```

For the dock data the statistic is about 0.95, and the promise holds. Do not expect that gift every time.

The turbine case: offsets

A wind operator in the Texas Panhandle runs 210 identical 2.3-MW turbines across two sites: a plains site with smooth, steady inflow, and a mesa site with complex terrain and turbulent inflow. Every unplanned maintenance event — a fault that stops the turbine and needs intervention — costs technician time, lost generation, and sometimes a crane day. The reliability team wants to know how fast the event rate grows with turbine age and whether the mesa really is harder on machines; the answers drive the parts budget and where the next repower happens. The data are one row per turbine per month for six months:

```
fleet <- read_csv(file.path(module_dir, "turbine_maintenance.csv")) |>
  mutate(Site = factor(Site, levels = c("plains", "mesa")))
fleet |> head(4)
```

```
# A tibble: 4 x 6
  TurbineID Month AgeYears Site   RunHours MaintEvents
  <chr>     <dbl>    <dbl> <fct>     <dbl>       <dbl>
1 WT001         1       10 plains      543           0
2 WT001         2       10 plains      550           1
3 WT001         3       10 plains      632           3
4 WT001         4       10 plains      595           0
```

One column changes everything: operating hours. A parked turbine accumulates no wear, mesa turbines run about 40% fewer hours than plains turbines (turbulence means more curtailment and more fault downtime), and older turbines run less than newer ones. Exposure differs across exactly the comparisons the question cares about.

The naive count model ignores this:

```r
turbine_naive <- glm(MaintEvents ~ AgeYears + Site, data = fleet, family = poisson())
tidy(turbine_naive, exponentiate = TRUE)
```

```
# A tibble: 3 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)    0.689   0.0690     -5.40  6.54e- 8
2 AgeYears       1.09    0.00651    12.6   1.16e-36
3 Sitemesa       1.01    0.0488      0.116 9.08e- 1
```

It reports a mesa rate ratio of 1.01 with $p = 0.91$: no site effect. That is a correct answer to the wrong question. The naive model describes events per *month*, and mesa turbines log no more events per month because they run so many fewer hours. The reliability question is about events per *operating hour*.

Writing the mean as a rate times an exposure shows the fix:

$$\mu_i = \text{rate}_i \times \frac{\text{Hours}_i}{1{,}000} \quad \Longrightarrow \quad \log(\mu_i) = \log\left(\frac{\text{Hours}_i}{1{,}000}\right) + \log(\text{rate}_i).$$

The exposure term enters the linear predictor with its coefficient pinned at 1. A pinned term is an **offset**, and with it in place every other coefficient describes the rate:

```r
turbine_offset <- glm(
  MaintEvents ~ AgeYears + Site + offset(log(RunHours / 1000)),
  data = fleet,
  family = poisson()
)
tidy(turbine_offset, exponentiate = TRUE, conf.int = TRUE)
```

```
# A tibble: 3 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)     1.03   0.0690      0.418 6.76e- 1    0.898      1.18
2 AgeYears        1.10   0.00652    14.7   9.11e-49    1.09       1.11
3 Sitemesa        1.69   0.0488     10.7   7.89e-27    1.53       1.86
```

Per hour of operation, the mesa site is about 69% harder on machines (rate ratio 1.69, CI roughly 1.53 to 1.86), and each year of age multiplies the per-hour rate by about 1.10. The naive model hid all of this behind the hours gap.

The turbine case: overdispersion and the negative binomial

The dock data hugged the variance-equals-mean line. The turbine data do not:

```r
sum(residuals(turbine_offset, type = "pearson")^2) / df.residual(turbine_offset)
```

```
[1] 2.173542
```

The dispersion statistic is about 2.17: the counts are twice as variable as the Poisson model claims. The reason is not mysterious. The model treats two turbine-months with the same age, site, and hours as identical, but turbines the model sees as identical are not: some drew a gearbox from a bad casting batch, micro-siting puts some units in dirtier air than their neighbors, and maintenance history differs unit to unit. Unmeasured differences pile extra variation on top of Poisson noise. That extra variation is **overdispersion**.

Overdispersion does not bias the rate ratios — the mean model is fine. It corrupts the *uncertainty*: Poisson standard errors here are roughly $\sqrt{2.17} \approx 1.5$ times too small, so every interval is too narrow and every p-value too confident.

The standard fix keeps the mean model and repairs the variance promise. The negative binomial, in its mean-based parametrization, says

$$E(Y_i) = \mu_i, \qquad \operatorname{Var}(Y_i) = \mu_i + \frac{\mu_i^2}{\theta}.$$

The new parameter θ is a dial for unmeasured heterogeneity: as $\theta \to \infty$ the extra term vanishes and the model collapses back to Poisson; small θ means lots of unit-to-unit variation the model cannot see. The link, the coefficients-as-rate-ratios reading, and the offset all carry over unchanged.

```r
library(MASS)

turbine_nb <- glm.nb(
  MaintEvents ~ AgeYears + Site + offset(log(RunHours / 1000)),
  data = fleet
)
tidy(turbine_nb, exponentiate = TRUE, conf.int = TRUE)
```

```
# A tibble: 3 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)     1.06   0.0964      0.582 5.61e- 1    0.877      1.27
2 AgeYears        1.10   0.00953     9.78  1.31e-22    1.08       1.12
3 Sitemesa        1.67   0.0712      7.21  5.52e-13    1.45       1.92
```

```r
turbine_nb$theta
```

```
[1] 1.33218
```

The estimated θ is about 1.33. The rate ratios barely move — mesa goes from 1.69 to 1.67 — but the standard errors grow to honest size (the site coefficient's SE rises from 0.049 to 0.071). Same answer, honest uncertainty.

In fleet terms: a 12-year mesa turbine runs about 3.9 times the per-hour event rate of a 3-year plains turbine. That curve prices the aging fleet, and it says the mesa site gets the next repower.

The GLM family

One grammar, three variance stories:

response	family	link	e^{β} means
yes/no	`binomial()`	logit	odds ratio
count	`poisson()`	log	rate ratio
overdispersed count	`glm.nb()`	log	rate ratio, honest SEs

Every model is a linear predictor, a link, and a distribution. The linear predictor is the same grammar as the whole course: dummies, adjustment variables, interactions, and now offsets. What changes across the family is only what kind of outcome sits on the left and what promise the model makes about its variance — and the variance promise is a claim to check, not a formality. Model the log of the mean, not the log of the data; put exposures in as offsets; and when the dispersion statistic drifts far above 1, switch to the negative binomial before quoting an interval.